

Samuel Oliveira

Ph0enix



Github



LinkedIn



Website



HoneyLab: Technical Report

Document information:

Title	HoneyLab: Technical Report
Client	Laboratory / Research
Author	Samuel Oliveira
Date	02/08/2026
Version	1.0
Classification	Confidential

Version control:

Version	Date	Author	Description
1.0	13/08/2026	Samuel Oliveira	Expanded version

Distribution:

Name	Role	Organisation
Samuel Oliveira	Author	Laboratory



Index

Introduction.....	5
What a Honeypot Is	6
Purpose and When It Is Useful	7
Scope and Authorisation.....	8
Base Environment and Operating System	8
Capture Components (Defence)	9
Cowrie: SSH and Telnet Honeypot.....	9
OpenCanary: FTP, HTTP and MySQL	10
SNARE and Tanner: Web Honeypot	11
Detection and Analysis Pipeline (ELK).....	12
Filebeat: Log Collection and Shipping	12
Logstash: GeoIP and MITRE ATT&CK Enrichment.....	13
Elasticsearch: Storage and Search	14
Kibana: Visualisation and Analysis	14
ElastAlert 2: Email Alerts	15
How Everything Connects: The Big Picture.....	16
Control Panel and Setup Wizard	17
Option 0: First-Time Setup Wizard	18
Options 1 to 4: Pages and Interfaces	19
Options 5 to 8: Real-Time Logs.....	19
Option 9: Detailed Service Status.....	19
pt / en: Change the Language	19
Hardening and Security	20
Cowrie: Resistance to nmap Detection.....	20
Tanner Isolation: No Docker Access	21
Unique Identity and Template Sanitisation	21
Known Limitations.....	22
Tanner Detection (Web Attacks).....	22
Tanner's Own Dashboard (Port 8091)	23
Cowrie vs. ssh-audit	23
Packaging and Publishing.....	24
Analysis Methodology	25
Final Recommendations	25
Conclusion.....	26





Introduction

This report documents, in technical detail and comprehensively, HoneyLab: a multi-service honeypot built on an Ubuntu Server 26.04 LTS virtual machine running on VMware. It is written to be readable both by someone who has never heard of honeypots and by a security engineer: it first explains what a honeypot is and what it is for, describes every defence and analysis component (what it is, how it works, which services it has and how it is configured), explains how everything connects end to end, details the management panel option by option, sets out the known limitations with full transparency, and ends with the packaging and publishing process.



What a Honeypot Is

A honeypot is a decoy system: a computer, service or application deliberately exposed and made to look vulnerable, whose sole purpose is to attract attackers and record everything they do. Unlike a real server, a honeypot has no legitimate users and serves no production role, so any interaction with it is by definition suspicious: there are no false positives from normal traffic. Everything that reaches it is reconnaissance, scanning or an attack.

Honeypots are classified by the level of interaction they offer the attacker. A low-interaction honeypot only simulates a service and logs the attempts (for example, it accepts an FTP connection and records the username and password tried, but gives no real session). A medium-interaction honeypot partially emulates the service, letting the attacker go further (for example, a fake shell where every command is logged). A high-interaction honeypot uses real systems, which yields very rich data but with a high risk of being used as a springboard for further attacks. HoneyLab combines low and medium interaction, the ideal balance between data richness and safety.

It is worth distinguishing an individual honeypot from a honeynet: a honeynet is an entire network of honeypots. HoneyLab is, in practice, a mini honeynet on a single machine, because it exposes several different decoy services (SSH, Telnet, FTP, HTTP, MySQL and a website) and centralises the analysis of all of them.



Purpose and When It Is Useful

A honeypot fulfils several security roles at once. The main one is to act as an early-detection system: since no one has a legitimate reason to touch it, the first sign that someone is probing the network often appears on the honeypot first, before the real systems are attacked.

- Early detection and alerting: it spots scans and intrusion attempts right at the start, acting as a silent alarm.
- Threat intelligence: it collects source IPs, the most-tried credentials, executed commands, downloaded tools and the tactics used, revealing who the real adversary is.
- Distraction and attacker attrition: while the attacker wastes time on the decoy, the real systems are safer and the team gains time to react.
- Research and training: it is a safe, isolated environment for studying malware, analysing attack campaigns and training security analysts, which is exactly the aim of this lab.
- Internal lateral-movement detection: placed inside the network, it exposes an attacker who has already broken in and is exploring machines, since no legitimate user would access it.

A honeypot is especially useful in research and training labs, in corporate networks as an internal alert sensor, and in any scenario where studying attacker behaviour matters without endangering production systems. It does not replace firewalls, antivirus or intrusion-detection systems: it complements them, adding visibility into the adversary's intent and techniques.



Scope and Authorisation

HoneyLab is intended for laboratory, training and research use. During development it runs on an isolated network (VMware NAT), with no direct Internet exposure. Production deployment or public exposure is only legitimate with written authorisation, a defined scope, an agreed time window and an emergency contact.

The management and analysis interfaces must never be exposed publicly. This includes the management OpenSSH service (port 2222), Kibana (port 5601), Elasticsearch (port 9200) and Logstash (ports 5044 and 9600). Only the decoy services should be reachable by the attacker.

Base Environment and Operating System

The system is based on a VMware virtual machine running Ubuntu Server 26.04 LTS. The management hostname is 'honeypot' and the primary network interface is 'ens32'. All honeypot services are managed by dedicated systemd units, which guarantees automatic start-up, restart on failure and privilege isolation via per-service users.

Administrative access is over real SSH (OpenSSH) on port 2222 with the 'superadmin' user. This separation (management on 2222, decoy on 22) is essential: port 22 is redirected by iptables to the Cowrie honeypot, so the administrator never uses port 22 for management. On first boot, a systemd service ('regenerate-ssh-host-keys.service') regenerates the SSH host keys, ensuring a unique cryptographic identity per deployment.

The VM is shipped as a ready-to-use template. The factory credentials are: user 'superadmin' with password 'honeypot01' (also used for sudo) and the Elasticsearch user 'elastic' with password 'changeme'. No alert email is configured. These credentials must all be changed on first use, through the panel wizard (option 0), as detailed later.



Capture Components (Defence)

The capture layer consists of three complementary engines, each covering distinct protocols and interaction levels. They are what the attacker sees and interacts with; everything they do is logged and forwarded to the analysis pipeline.

Cowrie: SSH and Telnet Honeypot

What it is: Cowrie (version 3.0.6, on top of the Twisted / Twisted Conch 26.4.0 library) is a medium-interaction honeypot for the SSH and Telnet protocols, the most common targets of brute-force attacks on the Internet.

How it works: when an attacker connects, Cowrie presents a banner and a login prompt identical to a real server's. It accepts (or rejects) credentials and, if it lets the attacker in, presents a full fake shell, with the hostname 'srv-storage-prod' and a fake filesystem where the attacker can browse, run simulated commands and download files. None of this touches the real system: it is all emulated. Every login attempt (username and password), every command entered, every transferred file (stored and hashed) and all session metadata are recorded in structured JSON.

- Services and ports: it listens on 2223 (SSH) and 2224 (Telnet), unprivileged ports. Via iptables (NAT/PREROUTING), traffic destined to 22 is redirected to 2223 and 23 to 2224, so the attacker sees the normal ports (22/23).
- Configuration, banner: it presents 'SSH-2.0-OpenSSH_9.2p1 Debian-2+deb12u3', to simulate a legitimate, up-to-date Debian server (set in cowrie.cfg).
- Configuration, execution: it runs as a systemd service ('cowrie'), under its own service user without root privileges; the install lives in '/home/cowrie/cowrie/' and the JSON logs ('cowrie.json') are consumed by Filebeat.
- Configuration, hardening: the SSH cryptographic algorithms were aligned with those of a modern server so Cowrie does not give itself away to nmap (see the hardening section).



OpenCanary: FTP, HTTP and MySQL

What it is: OpenCanary is a low-interaction honeypot that exposes several decoy services at once, each pretending to be a legitimate network service, in order to capture access and credential attempts.

How it works: for each enabled service, OpenCanary opens the matching port and responds just enough for the attacker to think it is real, immediately logging who connected, from where and with which credentials. It grants no real access to anything. The node identifier (`node_id`) was normalised to 'opencanary-01' and all events are written to '/var/log/opencanary.log'.

- FTP service (port 21): presents a fake file server and logs every authentication attempt (username and password).
- HTTP service (port 80): serves the login page of a fake NAS, with the brand/skin 'DS-STORAGE-01', and captures the credentials submitted in the form.
- MySQL service (port 3306): simulates a database server and logs connection and authentication attempts.
- Configuration and execution: defined in OpenCanary's configuration file (which services are enabled, ports and `node_id`); it runs from a dedicated Python virtual environment in '/opt/opencanary/venv', managed by systemd; the log is consumed by Filebeat.



SNARE and Tanner: Web Honeypot

What it is: the SNARE + Tanner pair is a web honeypot. SNARE is the server that presents a fake website to the attacker; Tanner is the brain that analyses each request and decides how to respond and how to classify it.

How it works: SNARE serves a fake website on port 8080 and forwards each HTTP request to Tanner. Tanner analyses the request, classifies the attack type (for example SQL Injection, XSS or RFI) and returns a coherent response, which SNARE delivers to the attacker. This makes the site look alive and reactive to attacks while logging and classifying them.

- Services and containers: Tanner runs on Docker across five containers: 'tanner' (analysis engine), 'tanner_api', 'tanner_web' (dashboard on port 8091), 'tanner_redis' (state and sessions) and 'tanner_phpox' (PHP execution sandbox). All containers restart automatically.
- Configuration, orchestration: the Tanner stack was relocated to '/opt/honeypot/tanner/' (with the docker-compose file at '/opt/honeypot/tanner/docker/docker-compose.yml'), outside the personal home folders, so that renaming the VM user never breaks the paths mounted into the containers.
- Configuration, SNARE: it runs as a systemd service ('snare.service'), under the 'superadmin' user, from '/home/superadmin/snare'; it points Tanner at 127.0.0.1. Because SNARE depends on Tanner (Docker), on boot it may fail until the containers are ready and then recover on its own through the systemd restart policy.
- Configuration, data flow: the attack classifications produced by Tanner are written to '/opt/honeypot/tanner/logs/tanner_report.json', which Filebeat reads and ships to Elasticsearch (the 'honeypot-snare-*' index). Tanner's detection limitations are detailed in the limitations section.



Detection and Analysis Pipeline (ELK)

Capturing attacks is only half the job; the other half is turning thousands of log lines into useful information. To do this, events from all honeypots are centralised in an ELK 8.x pipeline that handles collection, parsing, enrichment, indexing, visualisation and alerting.

Filebeat: Log Collection and Shipping

What it is: Filebeat is a lightweight log-collection agent (a 'shipper'). Its job is to watch log files and forward each new line to the next stage, reliably and with low resource use.

How it works: Filebeat continuously monitors the log files of the three honeypots and, whenever a new event appears, forwards it. It tags each source with an identifier field (for example 'honeypot') so that Logstash, downstream, knows which honeypot the event came from and applies the correct processing.

- Monitored services/inputs: Cowrie's JSON log ('/home/cowrie/cowrie/var/log/cowrie/cowrie.json'), OpenCanary's log ('/var/log/opencanary.log') and the SNARE/Tanner report ('/opt/honeypot/tanner/logs/tanner_report.json').
- Configuration: defined in '/etc/filebeat/filebeat.yml'; it runs as a systemd service and forwards events to Logstash on port 5044.



Logstash: GeoIP and MITRE ATT&CK Enrichment

What it is: Logstash is the pipeline's processing and enrichment engine. It receives the raw events, interprets them and adds context to them before they are stored.

How it works: Logstash parses each event, adds geolocation to the source IP via a GeoIP database (MaxMind GeoLite2), which reveals the country and city the attack came from, and applies an explicit mapping to MITRE ATT&CK techniques, tagging each event with the corresponding tactic and technique. Finally, it writes the enriched event to Elasticsearch.

- `cowrie.login.failed / login.success` -> T1110 (Brute Force, Credential Access).
- `cowrie.command.input` -> T1059 (Command and Scripting Interpreter, Execution).
- `cowrie.session.file_download` -> T1105 (Ingress Tool Transfer, Command and Control).
- Services and ports: a 'beats' input on port 5044 (where it receives from Filebeat) and a monitoring API on port 9600, both local only.
- Configuration and indexing: defined in `/etc/logstash/conf.d/01-honeypot.conf`; events are written to daily indices, `'honeypot-cowrie-*'`, `'honeypot-opencanary-*'` and `'honeypot-snare-*'`. This file holds the Elasticsearch password, which the wizard updates automatically when the password is changed.



Elasticsearch: Storage and Search

What it is: Elasticsearch is the search database where all events are stored and indexed. It is the storage heart of the ELK stack.

How it works: it indexes each event so as to allow near-instant searches and aggregations over millions of records (by IP, country, service, MITRE technique, date, etc.). It is on these indices that Kibana draws its charts and that ElastAlert 2 evaluates its alert rules.

- Services and ports: version 8.x, single node, with security and TLS enabled, exposed locally at 'https://localhost:9200'. Because TLS uses a self-signed certificate, local calls use the trust flag (curl -k).
- Configuration and credentials: the administrative user is 'elastic', with the default password 'changeme' (which must be changed via the wizard). The password change is done securely by the official 'elasticsearch-reset-password' tool.

Kibana: Visualisation and Analysis

What it is: Kibana is the graphical interface of the ELK stack, and HoneyLab's main analysis surface. It is where the analyst views, filters and interprets everything that was captured.

How it works: it connects to Elasticsearch and presents the data in interactive dashboards. It authenticates to Elasticsearch via a 'service account token' (not the 'elastic' user's password), which means the dashboards keep working even after the 'elastic' password is changed by the wizard.

- Services and ports: it listens on port 5601; it includes the 'honeypot-*' data view that aggregates the three indices.
- Configuration and content: it ships a pre-built operational dashboard ('Honeypot - Operational View') with a GeolP attacker map, top source IPs, most-attempted credentials, most-executed commands, distribution by MITRE ATT&CK technique and an event timeline.



ElastAlert 2: Email Alerts

What it is: ElastAlert 2 is the alerting engine. While Kibana is for active review, ElastAlert 2 watches the data continuously and notifies by email when it detects an attack pattern, without anyone having to be staring at the screen.

How it works: it runs as a service and periodically queries Elasticsearch for the patterns defined in its rules. When a rule fires, it sends an alert email via Gmail SMTP, using an App Password configured by the user in the panel wizard.

- `cowrie-bruteforce`: detects SSH brute force (for example, 5 or more failed logins from the same IP within a 5-minute window).
- `opencanary-login`: detects credential attempts against the OpenCanary services (FTP/HTTP/MySQL).
- `snare-attack`: detects web requests classified as an attack by Tanner.
- **Configuration**: it lives in `/etc/elastalert2/`; the `'config.yaml'` file holds the Elasticsearch connection and password (updated by the wizard), the rules are in their own files (`cowrie-bruteforce`, `opencanary-login`, `snare-attack`) and the email credentials are in `'smtp_auth.yaml'`. The recipient and the App Password are written by the wizard through the `'honeypot-set-email'` helper.



How Everything Connects: The Big Picture

The system works as a continuous chain, from the attacker's first packet to the alert email in the administrator's inbox. The path is as follows:

- 1. Capture: the attacker connects to a decoy service, SSH/Telnet (Cowrie), FTP/HTTP/MySQL (OpenCanary) or the website (SNARE, which consults Tanner). The service records the interaction in a local log file.
- 2. Collection: Filebeat watches those files and sends each new event to Logstash (port 5044).
- 3. Enrichment: Logstash parses it, adds geolocation (GeoIP) to the source IP and maps the event to a MITRE ATT&CK technique.
- 4. Storage: the enriched event is indexed in Elasticsearch, in the corresponding daily index ('honeypot-cowrie/opencanary/snare-*').
- 5. Visualisation: Kibana reads those indices and shows everything in dashboards (map, tops, techniques, timeline), the surface where analysis is done.
- 6. Alerting: in parallel, ElastAlert 2 continuously evaluates the same data and, when an attack pattern fires a rule, sends an alert email to the administrator.

This whole flow is local to the VM (analysis never leaves the machine); only the email alerts go outside. The control panel, described next, gives quick access to each piece of this chain.



Control Panel and Setup Wizard

On login (on the VM console or over SSH), a shell control panel is shown automatically. This is achieved by the shell profile (via `PROMPT_COMMAND`), which invokes the script installed at `'/usr/local/bin/honeypot-panel'`. At the top, the panel shows a header with the VM IP, date and time; if the initial setup is not yet complete, it also shows a red warning listing exactly what is left to configure.

Service status: just below, the panel shows the status (active/down, green or red) of each component, Cowrie, OpenCanary, SNARE, Kibana, Elasticsearch and the Tanner container, for an immediate health check. The panel is fully bilingual (Portuguese/English) and the language is saved per user in the `'~/.honeypot-lang'` file. A description of every option follows.



Option 0: First-Time Setup Wizard

Typing '0' (or 'setup') opens the first-run wizard. It detects the real state of each item and marks it as PENDING (red) or DONE (green, with the date of the last change). There are four items; each one asks for the administrator password before allowing changes, and in any submenu you can type '0' to go back. While any item is pending, the panel's red warning persists; when all four are done, the '/etc/honeypot-panel/.setup_done' flag is created.

- Item 1, Change the VM username: asks for the new username (it cannot be 'root', nor the factory name 'superadmin', nor an already existing one). Since it is not safe to rename a user with an open session, the change is scheduled for the next boot and the VM reboots; on return, you log in with the new username and the same password. A boot service renames the user and updates every reference (sudoers, SNARE, Filebeat, panel).
- Item 2, Change the VM password: asks for the new password (it cannot be empty nor the factory 'honeypot01') and applies it immediately to the current user.
- Item 3, Change the Elasticsearch password: asks for the new password (it cannot be the factory 'changeme'), resets it with the official tool and automatically propagates it to Logstash and ElastAlert 2, restarting those services for the change to take effect.
- Item 4, Configure the alert email: asks for the Gmail address where you want to receive alerts and its 16-character App Password (generated in the Google account). The wizard validates the email, validates the App Password format and performs a real SMTP login test against Gmail before saving; it only saves if the login succeeds. From then on, the email is bound to the three ElastAlert 2 rules and starts appearing in the wizard itself as 'Configured email'.



Options 1 to 4: Pages and Interfaces

These options show the address to open in your computer's (host) browser to reach each web interface (the panel does not open a browser inside the VM; it shows the URL, using the VM IP). Type the number and press Enter; to go back to the panel, press Enter.

- Option 1, View the fake site (SNARE): opens the decoy site at 'http://IP:8080', what the attacker sees.
- Option 2, Inspect the fake admin panel (OpenCanary): opens the fake NAS login page at 'http://IP:80'.
- Option 3, View the Kibana Dashboards: opens Kibana at 'http://IP:5601' (log in with 'elastic' and the Elasticsearch password). This is where analysis is done.
- Option 4, Tanner web analysis dashboard: opens the Tanner interface at 'http://IP:8091' (see, in the limitations, why the counters on this page stay at zero).

Options 5 to 8: Real-Time Logs

These options show the logs running live, directly in the terminal, to see what is happening right now. Type the number and press Enter; to exit and return to the panel, press Ctrl + C.

- Option 5, Cowrie logs: SSH / Telnet activity (logins and commands).
- Option 6, OpenCanary logs: FTP / HTTP / MySQL activity.
- Option 7, SNARE logs: requests to the fake site.
- Option 8, Tanner logs: analysis and classification of web attacks.

Option 9: Detailed Service Status

Typing '9' shows a detailed status: the (Active) state of each systemd service (Cowrie, OpenCanary, SNARE, Kibana, Elasticsearch) and the list of Tanner's Docker containers with their state. It is for quick diagnostics. Press Enter to return to the panel.

pt / en: Change the Language

Typing 'pt' or 'en' in the panel switches the whole interface (panel and wizard) to Portuguese or English. The choice is saved in the '~/.honeypot-lang' file and remembered in later sessions.



Hardening and Security

Cowrie: Resistance to nmap Detection

By default, Cowrie gives itself away: it advertises a modern OpenSSH banner but offers a set of old, inconsistent cryptographic algorithms (for example 3des-cbc, cast128-cbc, CBC-mode ciphers, hmac-sha1/hmac-md5 MACs and -sha1 key exchanges). Tools such as 'nmap' (the ssh2-enum-algos script) and 'ssh-audit' detect this inconsistency immediately and identify the emulation. To remove this signature, the offered algorithms were aligned with those of a modern, hardened server by editing the [ssh] section of 'cowrie.cfg':

- Ciphers: aes256-ctr, aes192-ctr, aes128-ctr (no CBC, 3DES or CAST128).
- MACs: hmac-sha2-512, hmac-sha2-256 (no hmac-sha1 or hmac-md5).
- Key exchange (kex): curve25519-sha256, curve25519-sha256@libssh.org, ecdh-sha2-nistp256/384/521 and diffie-hellman-group-exchange-sha256 (no SHA1 variants).
- Compression: none, zlib@openssh.com.

Since Cowrie did not natively expose the KEX option to the Twisted engine, a small patch to Cowrie's 'factory.py' was required so that it reads the '[ssh] kex' key. The result breaks Cowrie's classic signature and defeats an 'nmap -sV/'ssh2-enum-algos', with no loss of performance or capture capability (verified: logins and commands are still logged and reach Elasticsearch).



Tanner Isolation: No Docker Access

Some Tanner emulators (namely LFI, command injection and template injection) require access to the Docker socket to spawn auxiliary containers (Docker-in-Docker). In a honeypot, mounting the Docker socket into a container exposed to the attacker would be a critical risk: it would allow container escape and root privileges on the host. As a deliberate security decision, those emulators remain disabled, a functional limitation accepted in favour of host security.

Unique Identity and Template Sanitisation

So that each deployment of the template is distinct and shares no secrets, the packaging process includes the 'honeypot-sanitize-final.sh' script, which removes the SSH host keys, the machine-id, shell histories, the authorised and known SSH keys (authorized_keys, known_hosts), HSTS state, network Wi-Fi BSSIDs, cloud-init artefacts, the apt cache and all logs, leaving the system clean. The SSH host keys are then regenerated uniquely on first boot by the dedicated systemd service.



Known Limitations

For full transparency, this section describes what does not work at 100% and the exact technical causes.

Tanner Detection (Web Attacks)

Tanner reliably detects three attack classes, SQL Injection (SQLi), Cross-Site Scripting (XSS) and Remote File Inclusion (RFI), which are logged in Kibana with their classification (the 'honeypot-snare-*' index). The remaining types are not reliably detected, for the following technical reasons:

- LFI, command injection and template injection: their emulators require the Docker socket and are disabled for security (see the hardening section); enabling them can even cause Tanner to crash due to the missing access.
- PHP code injection: it is classified as XSS, because the '<?php ... ?>' payload contains the '<' and '>' characters and Tanner's (broader) XSS detection pattern (regex) takes precedence over the PHP one.
- CRLF injection: it is not reliably injectable via URL, because the HTTP layer sanitises the carriage-return and newline characters (\r\n).
- XXE (XML External Entity): it requires parsing an XML POST body, which Tanner does not perform on simple requests.
- PHP object injection: it depends on a specific payload format/encoding that generic test requests do not satisfy.



Tanner's Own Dashboard (Port 8091)

Tanner's per-session statistics page (port 8091) shows zero even when attacks are detected. The cause is version drift between the 'mushorg/tanner:latest' and 'mushorg/redis:latest' images: the redis client is incompatible (the 'encoding' argument was removed in recent versions), the emulator signatures are misaligned with 'base.py', and the session aggregator ('session_analyzer') does not populate the attack count field ('attack_count'). It is important to stress: attacks ARE detected and DO reach Kibana (with classification); real analysis is done in Kibana. The 8091 page is a secondary Tanner interface that became partially inoperative.

Cowrie vs. ssh-audit

Against 'nmap', detection is solved. Against a dedicated 'ssh-audit', Cowrie is not 100% indistinguishable from a real server: the Twisted library (used by Cowrie) does not implement the most modern algorithms (chacha20-poly1305, AES-GCM ciphers) nor advertise 'rsa-sha2-*' as a host key algorithm. A careful analyst notes a 'hardened server, but without the modern AEAD ciphers'. Closing this gap would require a risky patch to the Twisted library or placing a real OpenSSH front-end (in a man-in-the-middle configuration), both options were rejected as they would compromise the stability and reliability of the capture.



Packaging and Publishing

Once the VM was configured and tested, it was prepared for distribution as a ready-to-use template. The process had three phases: sanitisation, export and distribution/publishing.

Sanitisation and export: with the VM running, 'honeypot-sanitize-final.sh' was executed (it wipes SSH host keys, machine-id, histories, logs and other traces, leaving the system in a factory state) and the machine was shut down cleanly. In VMware, with the VM off, snapshots were consolidated/deleted, the '.vmdk' disk was Compacted (to reclaim free space) and the VM was exported to a single OVA file ('HoneyLab.ova'). Important note: no aggressive zero-fill with 'dd' is used before Compact, because it can crash the VMware Workstation I/O manager; Compact itself already reclaims the free blocks.

Single-file distribution: the exported OVA is about 5.7 GB and, because the disk inside the OVA is already compressed, additional compression is practically irrelevant (recompressing shrinks it by only about 1%). Therefore, rather than splitting the VM into several volumes, 'HoneyLab.ova' is distributed as a single file through a free cloud service (MEGA), whose free tier (20 GB) imposes no 2 GB per-file limit. Optionally, the same OVA is also provided inside a single '.zip' archive (which shrinks it by only about 1% but gives an automatic integrity check on extraction). A SHA256 checksum of the file is also published so the user can confirm the download is not corrupted.

Publishing: the GitHub repository hosts the source code (panel, wizard, helpers and scripts), the usage guide (README) and this technical report. The VM itself ('HoneyLab.ova', ~5.7 GB) is provided via an external free-cloud link (MEGA), as it exceeds GitHub's 100 MB per-file limit. To use it, the user downloads 'HoneyLab.ova' (or 'HoneyLab.zip' and extracts it), optionally verifies the SHA256, imports the OVA into VMware (network in NAT for the lab) and starts the VM.



Analysis Methodology

To analyse a captured incident, the following methodology is recommended: (1) in Kibana, filter by source IP, service or MITRE ATT&CK technique; (2) reconstruct the full session timeline (login, command sequence, downloads); (3) correlate with geolocation (GeoIP) and user-agent; (4) for web attacks, consult the classification assigned by Tanner in the 'honeypot-snare-*' index; (5) validate and contextualise with the alerts generated by ElastAlert 2. It is recommended to keep at least 7 days of data to obtain meaningful metrics and trends.

Final Recommendations

- Complete the first-run wizard (option 0) before any real use: change the VM username, change the VM password, change the Elasticsearch password and configure the alert email.
- Never expose to the Internet the management ports (SSH 2222), Kibana (5601), Elasticsearch (9200) or Logstash (5044/9600); only the decoy services should be reachable.
- Periodically review the Kibana dashboards and the ElastAlert 2 alerts; define index rotation and retention policies.
- For web attack analysis, use Kibana and not Tanner's 8091 page.
- Never grant the Tanner container (or any decoy service) access to the Docker socket.
- Before exporting as a template, always run 'honeypot-sanitize-final.sh' and shut the VM down cleanly; do not boot the VM again before exporting it.



Conclusion

HoneyLab implements a complete and functional chain of capture (Cowrie, OpenCanary, SNARE/Tanner), detection, enrichment (GeoIP and MITRE ATT&CK), visualisation (Kibana) and alerting (ElastAlert 2), with effective hardening of Cowrie against nmap detection and a custom, bilingual management panel with a setup wizard. The existing limitations are documented with full transparency and stem mostly from deliberate security restrictions (Docker isolation) or version drift in third-party components (Tanner/Redis), and do not affect the core capture and analysis capability, which is robustly ensured by the ELK/Kibana pipeline.

Obrigado

Ph0enix



Github



LinkedIn



Website